



KTU NOTES

The learning companion.

**KTU STUDY MATERIALS | SYLLABUS | LIVE
NOTIFICATIONS | SOLVED QUESTION PAPERS**

Website: www.ktunotes.in

Module – 4 (Mobile Network and Transport Layer) Mobile network layer – Mobile Internet Protocol (IP), Dynamic Host Configuration Protocol (DHCP), Mobile ad-hoc networks – Routing, Dynamic Source Routing (DSR), Destination Sequence Distance Vector (DSDV), Ad-hoc routing protocols. Mobile transport layer – Traditional Transmission Control Protocol (TCP), Improvements in Classical TCP. Wireless Application Protocol (WAP) - Architecture, Wireless Datagram Protocol (WDP), Wireless Transport Layer Security (WTLS), Wireless Transaction Protocol (WTP), Wireless Session Protocol (WSP).

Mobile IP

Goals, assumptions and requirements

The internet is the network for global data communication with hundreds of millions of users. So why not simply use a mobile computer in the internet? The reason is quite simple: you will not receive a single packet as soon as you leave your home network, i.e., the network your computer is configured for, and reconnect your computer (wireless or wired) at another place (if no additional mechanisms are available). The reason for this is quite simple if you consider routing mechanisms on the internet. A host sends an IP packet with the header containing a destination address with other fields. The destination address not only determines the receiver of the packet, but also the physical subnet of the receiver. For example, the destination address 129.13.42.99 shows that the receiver must be connected to the physical subnet with the network prefix 129.13.42 (unless CIDR is used, RFC 1519, Fuller, 1993). Routers in the internet now look at the destination addresses of incoming packets and forward them according to internal look-up tables. To avoid an explosion of routing tables, only prefixes are stored and further optimizations are applied. A router would otherwise have to store the addresses of all computers in the internet, which is obviously not feasible. As long as the receiver can be reached within its physical subnet, it gets the packets; as soon as it moves outside the subnet, a packet will not reach it. A host needs a so-called **topologically correct address**.

Quick ‘solutions’

One might think that a quick solution to this problem would be to assign to the computer a new, topologically correct IP address. This is what many users do with the help of DHCP. So moving to a new location would mean assigning a new IP address. The problem is that nobody knows about this new address. It is almost impossible to find a (mobile) host on the internet which has just changed its address.

Requirements

Since the quick ‘solutions’ obviously did not work, a more general architecture had to be designed. Many field trials and proprietary systems finally led to mobile IP as a standard to enable mobility in the internet. Several requirements accompanied the development of the standard:

- **Compatibility:** The installed base of Internet computers, i.e., computers running TCP/IP and connected to the internet, is huge. A new standard cannot introduce changes for applications or network protocols already in use. People still want to use their favorite browser for www and do not want to change applications just for mobility, the same holds for operating systems. Mobile IP has to be integrated into existing operating systems or at least work with them (today it is available for many platforms). Routers within the internet should not necessarily require other software. While it is possible to enhance the capabilities of some routers to support mobility, it is almost impossible to change all of them. Mobile IP has to remain compatible with all lower layers used for the standard, non-mobile, IP. Mobile IP must not require special media or MAC/LLC protocols, so it must use the same interfaces and mechanisms to access the lower layers as IP does. Finally, end-systems enhanced with a mobile IP implementation should still be able to communicate with fixed systems without mobile IP. Mobile IP has to ensure that users can still access all the other servers and systems in the internet. But that implies using the same address format and routing mechanisms.
- **Transparency:** Mobility should remain ‘invisible’ for many higher layer protocols and applications. Besides maybe noticing a lower bandwidth and some interruption in service, higher layers should continue to work even if the mobile computer has changed its point of attachment to the network. For TCP this means that the computer must keep its IP address as explained above. If the interruption of the connectivity does not take too long, TCP connections survive the change of the attachment point. Problems related to the performance of TCP are discussed in chapter 9. Clearly, many of today’s applications have not been designed for use in mobile environments, so the only effects of mobility should be a higher delay and lower bandwidth. However, there are some applications for which it is better to be ‘mobility aware’. Knowing that it is currently possible to use different networks, the software could choose the cheapest one. Or if a video application knows that only a low bandwidth connection is currently available, it could use a different compression scheme. Additional mechanisms are necessary to inform these applications about mobility.
- **Scalability and efficiency:** Introducing a new mechanism to the internet must

not jeopardize its efficiency. Enhancing IP for mobility must not generate too many new messages flooding the whole network. Special care has to be taken considering the lower bandwidth of wireless links. Many mobile systems will have a wireless link to an attachment point, so only some additional packets should be necessary between a mobile system and a node in the network. Looking at the number of computers connected to the internet and at the growth rates of mobile communication, it is clear that myriad devices will participate in the internet as mobile components. Just think of cars, trucks, mobile phones, every seat in every plane around the world etc. – many of them will have some IP implementation inside and move between different networks and require mobile IP. It is crucial for a mobile IP to be scalable over a large number of participants in the whole internet, worldwide.

- **Security:** Mobility poses many security problems. The minimum requirement is that of all the messages related to the management of Mobile IP are authenticated. The IP layer must be sure that if it forwards a packet to a mobile host that this host receives the packet. The IP layer can only guarantee that the IP address of the receiver is correct. There are no ways of preventing fake IP addresses or other attacks.

The goal of a mobile IP can be summarized as: ‘supporting end-system mobility while maintaining scalability, efficiency, and compatibility in all respects with existing applications and Internet protocols’.

Entities and terminology

- **Mobile node (MN):** A mobile node is an end-system or router that can change its point of attachment to the internet using mobile IP. The MN keeps its IP address and can continuously communicate with any other system in the internet as long as link-layer connectivity is given. Mobile nodes are not necessarily small devices such as laptops with antennas or mobile phones; a router onboard an aircraft can be a powerful mobile node.
- **Correspondent node (CN):** At least one partner is needed for communication. In the following the CN represents this partner for the MN. The CN can be a fixed or mobile node.

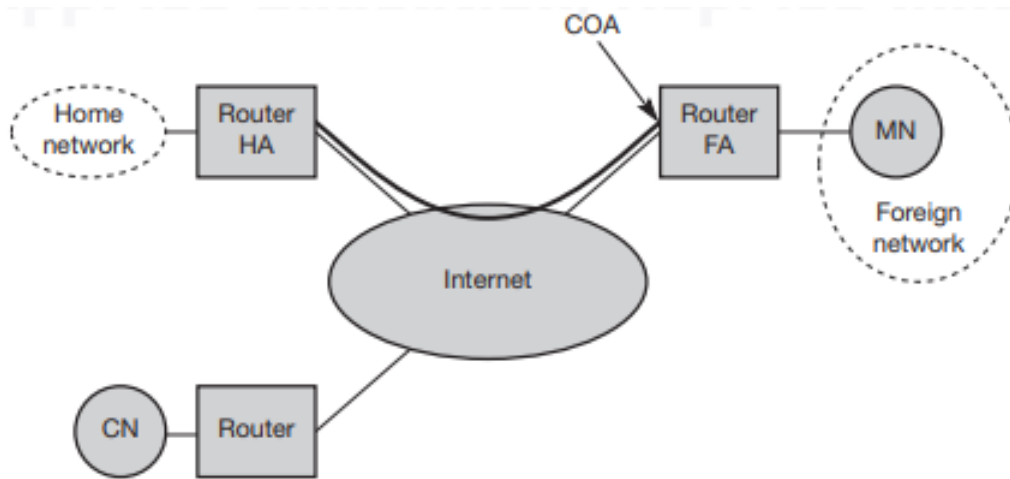


Fig.4.1 Mobile IP example network

- **Home network:** The home network is the subnet the MN belongs to with respect to its IP address. No mobile IP support is needed within the home network.
- **Foreign network:** The foreign network is the current subnet the MN visits and which is not the home network.
- **Foreign agent (FA):** The FA can provide several services to the MN during its visit to the foreign network. The FA can have the COA, acting as tunnel endpoint and forwarding packets to the MN. The FA can be the default router for the MN. FAs can also provide security services because they belong to the foreign network as opposed to the MN which is only visiting. For mobile IP functioning, FAs are not necessarily needed. Typically, an FA is implemented on a router for the subnet the MN attaches to.
- **Care-of address (COA):** The COA defines the current location of the MN from an IP point of view. All IP packets sent to the MN are delivered to the COA, not directly to the IP address of the MN. Packet delivery toward the MN is done using a tunnel, as explained later. To be more precise, the COA marks the tunnel endpoint, i.e., the address where packets exit the tunnel. There are two different possibilities for the location of the COA:
 - **Foreign agent COA:** The COA could be located at the FA, i.e., the COA is an IP address of the FA. The FA is the tunnel end-point and forwards packets to the MN. Many MN using the FA can share this COA as common COA.
 - **Co-located COA:** The COA is co-located if the MN temporarily acquired an additional IP address which acts as COA. This address is now topologically correct, and the tunnel endpoint is at the MN. Co-located addresses can be acquired using

services such as DHCP. One problem associated with this approach is the need for additional addresses if MNs request a COA. This is not always a good idea considering the scarcity of IPv4 addresses.

- **Home agent (HA):** The HA provides several services for the MN and is located in the home network. The tunnel for packets toward the MN starts at the HA. The HA maintains a location registry, i.e., it is informed of the MN's location by the current COA. Three alternatives for the implementation of an HA exist.
 - The HA can be implemented on a router that is responsible for the home network. This is obviously the best position, because without optimizations to mobile IP, all packets for the MN have to go through the router anyway.
 - If changing the router's software is not possible, the HA could also be implemented on an arbitrary node in the subnet. One disadvantage of this solution is the double crossing of the router by the packet if the MN is in a foreign network. A packet for the MN comes in via the router; the HA sends it through the tunnel which again crosses the router.
 - Finally, a home network is not necessary at all. The HA could be again on the 'router' but this time only acting as a manager for MNs belonging to a virtual home network. All MNs are always in a foreign network with this solution.

IP packet delivery

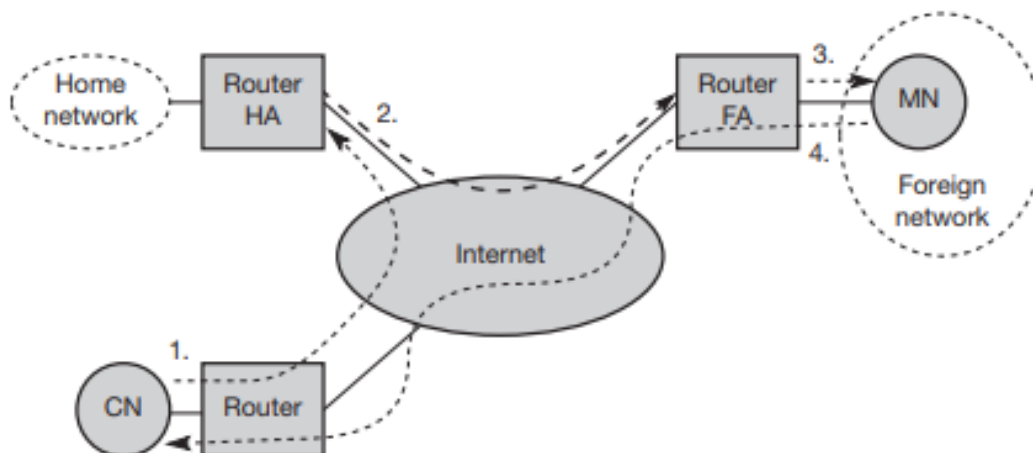


Fig.4.2 Packet delivery to and from the mobile node

Figure 4.2 illustrates packet delivery to and from the MN using the example net-

work of Figure 4.1. A correspondent node CN wants to send an IP packet to the MN. One of the requirements of mobile IP was to support hiding the mobility of the MN. CN does not need to know anything about the MN's current location and sends the packet as usual to the IP address of MN (step 1). This means that CN sends an IP packet with MN as a destination address and CN as a source address. The internet, not having information on the current location of MN, routes the packet to the router responsible for the home network of MN. This is done using the standard routing mechanisms of the internet.

The HA now intercepts the packet, knowing that MN is currently not in its home network. The packet is not forwarded into the subnet as usual, but encapsulated and tunnelled to the COA. A new header is put in front of the old IP header showing the COA as new destination and HA as source of the encapsulated packet (step 2). The foreign agent now decapsulates the packet, i.e., removes the additional header, and forwards the original packet with CN as source and MN as destination to the MN (step 3). Again, for the MN mobility is not visible. It receives the packet with the same sender and receiver address as it would have done in the home network.

At first glance, sending packets from the MN to the CN is much simpler; The MN sends the packet as usual with its own fixed IP address as source and CN's address as destination (step 4). The router with the FA acts as default router and forwards the packet in the same way as it would do for any other node in the foreign network. As long as CN is a fixed node the remainder is in the fixed internet as usual. If CN were also a mobile node residing in a foreign network, the same mechanisms as described in steps 1 through 3 would apply now in the other direction.

Dynamic host configuration protocol

The dynamic host configuration protocol is mainly used to simplify the installation and maintenance of networked computers. If a new computer is connected to a network, DHCP can provide it with all the necessary information for full system integration into the network, e.g., addresses of a DNS server and the default router, the subnet mask, the domain name, and an IP address. Providing an IP address, makes DHCP very attractive for mobile IP as a source of care-of-addresses. While the basic DHCP mechanisms are quite simple, many options are available.

DHCP is based on a client/server model as shown in Figure 4.3. DHCP clients send a request to a server (DHCPDISCOVER in the example) to which the server responds. A client sends requests using MAC broadcasts to reach all devices in the LAN. A DHCP relay might be needed to forward requests across inter-working units

to a DHCP server.

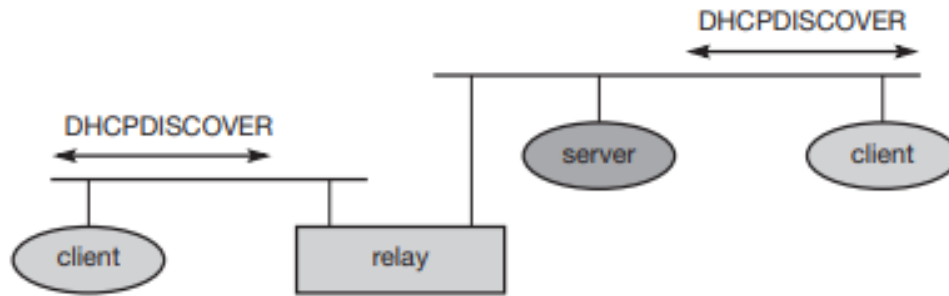


Fig.4.3 Basic DHCP configuration

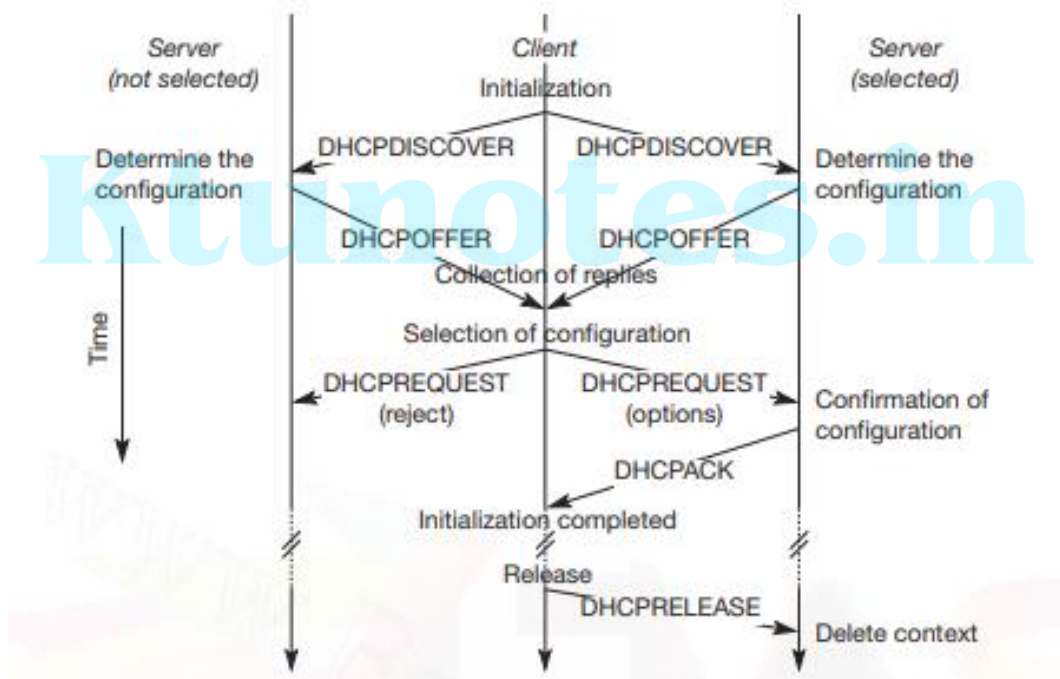


Fig.4.4 Client initialization via DHCP

A typical initialization of a DHCP client is shown in Figure 4.4. The figure shows one client and two servers. As described above, the client broadcasts a DHCPDISCOVER into the subnet. There might be a relay to forward this broadcast. In the case shown, two servers receive this broadcast and determine the configuration they can offer to the client. One example for this could be the checking of available IP addresses and choosing one for the client. Servers reply

to the client's request with DHCPOFFER and offer a list of configuration parameters. The client can now choose one of the configurations offered. The client in turn replies to the servers, accepting one of the configurations and rejecting the others using DHCPREQUEST. If a server receives a DHCPREQUEST with a rejection, it can free the reserved configuration for other possible clients. The server with the configuration accepted by the client now confirms the configuration with DHCPACK. This completes the initialization phase.

If a client leaves a subnet, it should release the configuration received by the server using DHCPRELEASE. Now the server can free the context stored for the client and offer the configuration again. The configuration a client gets from a server is only leased for a certain amount of time, it has to be reconfirmed from time to time. Otherwise the server will free the configuration. This time-out of configuration helps in the case of crashed nodes or nodes moved away without releasing the context.

DHCP is a good candidate for supporting the acquisition of care-of-addresses for mobile nodes. The same holds for all other parameters needed, such as addresses of the default router, DNS servers, the timeserver etc. A DHCP server should be located in the subnet of the access point of the mobile node, or at least a DHCP relay should provide forwarding of the messages. RFC 3118 (Drohms, 2001) specifies authentication for DHCP messages which is needed to protect mobile nodes from malicious DHCP servers. Without authentication, the mobile node cannot trust a DHCP server, and the DHCP server cannot trust the mobile node.

Mobile ad-hoc networks

Mobile IP requires, e.g., a home agent, tunnels, and default routers. DHCP requires servers and broadcast capabilities of the medium reaching all participants or relays to servers. However, there may be several situations where users of a network cannot rely on an infrastructure, it is too expensive, or there is none at all. In these situations mobile ad-hoc networks are the only choice. It is important to note that this section focuses on so-called multi-hop ad-hoc networks when describing ad-hoc networking. The ad-hoc setting up of a connection with an infrastructure is not the main issue here. These networks should be mobile and use wireless communications. Examples for the use of such mobile, wireless, multi-hop ad-hoc networks, which are only called ad-hoc networks here for simplicity, are:

- **Instant infrastructure:** Unplanned meetings, spontaneous interpersonal communications etc. cannot rely on any infrastructure. Infrastructures need planning and administration. It would take too long to set up this kind of infrastructure; therefore, ad-hoc connectivity has to be set up.
- **Disaster relief:** Infrastructures typically break down in disaster areas. Hurricanes cut phone and power lines, floods destroy base stations, fires burn servers. Emergency teams can only rely on an infrastructure they can set up themselves. No forward planning can be done, and the set-up must be extremely fast and reliable. The same applies to many military activities, which is, to be honest, one of the major driving forces behind mobile ad-hoc networking research.
- **Remote areas:** Even if infrastructures could be planned ahead, it is some- times too expensive to set up an infrastructure in sparsely populated areas. Depending on the communication pattern, ad-hoc networks or satellite infrastructures can be a solution.
- **Effectiveness:** Services provided by existing infrastructures might be too expensive for certain applications. If, for example, only connection- oriented cellular networks exist, but an application sends only a small status information every other minute, a cheaper ad-hoc packet-oriented network might be a better solution. Registration procedures might take too long, and communication overheads might be too high with existing networks. Application-tailored ad-hoc networks can offer a better solution.

Routing

While in wireless networks with infrastructure support a base station always reaches all mobile nodes, this is not always the case in an ad-hoc network. A destination node might be out of range of a source node transmitting packets. Routing is needed to find a path between source and destination and to forward the packets appropriately. In wireless networks using an infrastructure, cells have been defined. Within a cell, the base station can reach all mobile nodes without routing via a broadcast. In the case of ad-hoc networks, each node must be able to forward data for other nodes. This creates many additional problems that are discussed in the following paragraphs.

Figure 4.5 gives a simple example of an ad-hoc network. At a certain time t_1 the network topology might look as illustrated on the left side of the figure. Five nodes, N_1 to N_5 , are connected depending on the current transmission characteristics between them. In this snapshot of the network, N_4 can receive N_1 over a good link, but N_1 receives N_4 only via a weak link. Links do not necessarily have the same characteristics in both directions. The reasons for this are, e.g., different antenna

characteristics or transmit power. N_1 cannot receive N_2 at all, N_2 receives a signal from N_1 .

This situation can change quite fast as the snapshot at t_2 shows. N_1 cannot receive N_4 any longer, N_4 receives N_1 only via a weak link. But now N_1 has an asymmetric but bi-directional link to N_2 that did not exist before.

This very simple example already shows some fundamental differences between wired networks and ad-hoc wireless networks related to routing.

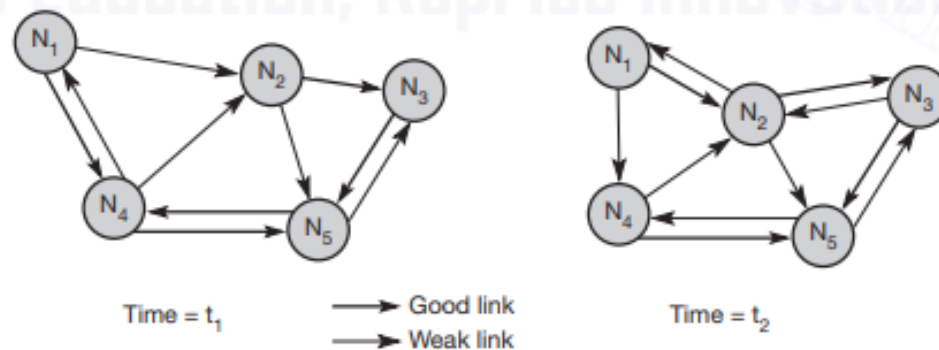


Fig.4.5 Examples of Adhoc Networks

- **Asymmetric links:** Node A receives a signal from node B. But this does not tell us anything about the quality of the connection in reverse. B might receive nothing, have a weak link, or even have a better link than the reverse direction. Routing information collected for one direction is of almost no use for the other direction. However, many routing algorithms for wired networks rely on a symmetric scenario.
- **Redundant links:** Wired networks, too, have redundant links to survive link failures. However, there is only some redundancy in wired networks, which, additionally, are controlled by a network administrator. In ad-hoc networks nobody controls redundancy, so there might be many redundant links up to the extreme of a completely meshed topology. Routing algorithms for wired networks can handle some redundancy, but a high redundancy can cause a large computational overhead for routing table updates.
- **Interference:** In wired networks links exist only where a wire exists, and connections are planned by network administrators. This is not the case for wireless ad-hoc networks. Links come and go depending on transmission characteristics, one transmission might interfere with another, and nodes might overhear the transmissions of other nodes. Interference creates new problems by ‘unplanned’ links between nodes:

if two close-by nodes forward two transmissions, they might interfere and destroy each other. On the other hand, interference might also help routing. A node can learn the topology with the help of packets it has overheard.

- **Dynamic topology:** The greatest problem for routing arises from the highly dynamic topology. The mobile nodes might move as shown in Figure 4.5 or medium characteristics might change. This results in frequent changes in topology, so snapshots are valid only for a very short period of time. In ad-hoc networks, routing tables must somehow reflect these frequent changes in topology, and routing algorithms have to be adapted. Routing table updates in fixed networks, for example, take place every 30 seconds. This updating frequency might be too low to be useful for ad-hoc networks. Some algorithms rely on a complete picture of the whole network. While this works in wired networks where changes are rare, it fails completely in ad-hoc networks. The topology changes during the distribution of the ‘current’ snapshot of the network, rendering the snapshot useless.

Let us go back to the example network in Figure 4.5 and assume that node N_1 wants to send data to N_3 and needs an acknowledgement. If N_1 had a complete overview of the network at time t_1 , which is not always the case in ad-hoc networks, it would choose the path N_1, N_2, N_3 , for this requires only two hops (if we use hops as metric). Acknowledgements cannot take the same path, N_3 chooses N_3, N_5, N_4, N_1 . This takes three hops and already shows that routing also strongly influences the function of higher layers. TCP, for example, makes round trip measurements assuming the same path in both directions. This is obviously wrong in the example shown, leading to misinterpretations of measurements and inefficiencies.

Just a moment later, at time t_2 , the topology has changed. Now N_3 cannot take the same path to send acknowledgements back to N_1 , while N_1 can still take the old path to N_3 . Although already more complicated than fixed networks, this example still assumes that nodes can have a complete insight into the current situation. The optimal knowledge for every node would be a description of the current connectivity between all nodes, the expected traffic flows, capacities of all links, delay of each link, and the computing and battery power of each node. While even in fixed networks traffic flows are not exactly predictable, for ad-hoc networks link capacities are additionally unknown. The capacity of each link can change from 0 to the maximum of the transmission technology used. In real ad-hoc networks no node knows all these factors, and establishing up-to-date snapshots of the network is almost impossible.

Dynamic source routing

Imagine what happens in an ad-hoc network where nodes exchange packets from time to time, i.e., the network is only lightly loaded, and DSDV or one of the traditional distance vector or link state algorithms is used for updating routing tables. Although only some user data has to be transmitted, the nodes exchange routing information to keep track of the topology. These algorithms maintain routes between all nodes, although there may currently be no data exchange at all. This causes unnecessary traffic and prevents nodes from saving battery power.

Dynamic source routing (DSR), therefore, divides the task of routing into two separate problems

- **Route discovery:** A node only tries to discover a route to a destination if it has to send something to this destination and there is currently no known route.
- **Route maintenance:** If a node is continuously sending packets via a route, it has to make sure that the route is held upright. As soon as a node detects problems with the current route, it has to find an alternative.

Dynamic source routing eliminates all periodic routing updates and works as follows. If a node needs to discover a route, it broadcasts a route request with a unique identifier and the destination address as parameters. Any node that receives a route request does the following.

- If the node has already received the request (which is identified using the unique identifier), it drops the request packet.
- If the node recognizes its own address as the destination, the request has reached its target.
- Otherwise, the node appends its own address to a list of traversed hops in the packet and broadcasts this updated route request.

Using this approach, the route request collects a list of addresses representing a possible path on its way towards the destination. As soon as the request reaches the destination, it can return the request packet containing the list to the receiver using this list in reverse order. One condition for this is that the links work bi-directionally. If this is not the case, and the destination node does not currently maintain a route back to the initiator of the request, it has to start a route discovery by itself. The destination may receive several lists containing different paths from the initiator. It could return the best path, the first path, or several paths

to offer the initiator a choice.

Applying route discovery to the example in Figure 4.5 for a route from N_1 to N_3 at time t_1 results in the following.

- N_1 broadcasts the request $((N_1), id = 42, target = N_3)$, N_2 and N_4 receive this request.
- N_2 then broadcasts $((N_1, N_2), id = 42, target = N_3)$, N_4 broadcasts $((N_1, N_4), id = 42, target = N_3)$. N_3 and N_5 receive N_2 's broadcast, N_1 , N_2 , and N_5 receive N_4 's broadcast.
- N_3 recognizes itself as target, N_5 broadcasts $((N_1, N_2, N_5), id = 42, target = N_3)$. N_3 and N_4 receive N_5 's broadcast. N_1 , N_2 , and N_5 drop N_4 's broadcast packet, because they all recognize an already received route request (and N_2 's broadcast reached N_5 before N_4 's did).
- N_4 drops N_5 's broadcast, N_3 recognizes (N_1, N_2, N_5) as an alternate, but longer route.
- N_3 now has to return the path (N_1, N_2, N_3) to N_1 . This is simple assuming symmetric links working in both directions. N_3 can forward the information using the list in reverse order.

The assumption of bi-directional links holds for many ad-hoc networks. However, if links are not bi-directional, the scenario gets more complicated. The algorithm has to be applied again, in the reverse direction if the target does not maintain a current path to the source of the route request.

- N_3 has to broadcast a route request $((N_3), id = 17, target = N_1)$. Only N_5 receives this request.
- N_5 now broadcasts $((N_3, N_5), id = 17, target = N_1)$, N_3 and N_4 receive the broadcast.
- N_3 drops the request because it recognizes an already known id. N_4 broadcasts $((N_3, N_5, N_4), id = 17, target = N_1)$, N_5 , N_2 , and N_1 receive the broadcast.
- N_5 drops the request packet, N_1 recognizes itself as target, and N_2 broadcasts $((N_3, N_5, N_4, N_2), id = 17, target = N_1)$. N_3 and N_5 receive N_2 's broadcast.
- N_3 and N_5 drop the request packet.

Now N_3 holds the list for a path from N_1 to N_3 , (N_1, N_2, N_3) , and N_1 knows

the path from N_3 to N_1 , (N_3, N_5, N_4, N_1). But N_1 still does not know how to send data to N_3 ! The only solution is to send the list (N_1, N_2, N_3) with the broadcasts initiated by N_3 in the reverse direction. This example shows clearly how much simpler routing can be if links are symmetrical.

The basic algorithm for route discovery can be optimized in many ways.

- To avoid too many broadcasts, each route request could contain a counter. Every node rebroadcasting the request increments the counter by one. Knowing the maximum network diameter (take the number of nodes if nothing else is known), nodes can drop a request if the counter reaches this number.
- A node can cache path fragments from recent requests. These fragments can now be used to answer other route requests much faster (if they still reflect the topology!).
- A node can also update this cache from packet headers while forwarding other packets.
- If a node overhears transmissions from other nodes, it can also use this information for shortening routes.

After a route has been discovered, it has to be maintained for as long as the node sends packets along this route. Depending on layer two mechanisms, different approaches can be taken:

- If the link layer uses an acknowledgement (as, for example, IEEE 802.11) the node can interpret this acknowledgement as an intact route.
- If possible, the node could also listen to the next node forwarding the packet, so getting a passive acknowledgement.
- A node could request an explicit acknowledgement.

Destination sequence distance vector

Destination sequence distance vector (DSDV) routing is an enhancement to distance vector routing for ad-hoc networks (Perkins, 1994). DSDV can be considered historically, however, an on-demand version (ad-hoc on-demand distance vector, AODV) is among the protocols currently discussed. Distance vector routing is used as routing information protocol (RIP) in wired networks. It performs extremely poorly with certain network changes due to the count-to-infinity problem. Each node exchanges its neighbor table periodically with its neighbors. Changes at one node in the network propagate slowly through the network. The strategies to avoid this problem which are used in fixed networks (poisoned-reverse/split-horizon) do not help in the case of wireless ad-hoc networks, due to the rapidly changing topology. This

might create loops or unreachable regions within the network.

DSDV now adds two things to the distance vector algorithm:

- **Sequence numbers:** Each routing advertisement comes with a sequence number. Within ad-hoc networks, advertisements may propagate along many paths. Sequence numbers help to apply the advertisements in correct order. This avoids the loops that are likely with the unchanged distance vector algorithm.
- **Damping:** Transient changes in topology that are of short duration should not destabilize the routing mechanisms. Advertisements containing changes in the topology currently stored are therefore not disseminated further. A node waits with dissemination if these changes are probably unstable. Waiting time depends on the time between the first and the best announcement of a path to a certain destination.

Destination	Next hop	Metric	Sequence no.	Instal time
N_1	N_1	0	S_1-321	T_4-001
N_2	N_2	1	S_2-218	T_4-001
N_3	N_2	2	S_3-043	T_4-002
N_4	N_4	1	S_4-092	T_4-001
N_5	N_4	2	S_5-163	T_4-002

Part of a routing table for DSDV

The routing table for N_1 in Figure 8.20 would be as shown in Table. For each node N_1 stores the next hop toward this node, the metric (here number of hops), the sequence number of the last advertisement for this node, and the time at which the path has been installed first. The table contains flags and a settling time helping to decide when the path can be assumed stable. Router advertisements from N_1 now contain data from the first, third, and fourth column: destination address, metric, and sequence number. Besides being loop-free at all times, DSDV has low memory requirements and a quick convergence via triggered updates.

Mobile transport layer

Traditional TCP

Congestion control

A transport layer protocol such as TCP has been designed for fixed networks with fixed end-systems. Congestion may appear from time to time even in carefully designed networks. The packet buffers of a router are filled and the router cannot

forward the packets fast enough because the sum of the input rates of packets destined for one output link is higher than the capacity of the output link. The only thing a router can do in this situation is to drop packets. A dropped packet is lost for the transmission, and the receiver notices a gap in the packet stream. Now the receiver does not directly tell the sender which packet is missing, but continues to acknowledge all in-sequence packets up to the missing one.

The sender notices the missing acknowledgement for the lost packet and assumes a packet loss due to congestion. Retransmitting the missing packet and continuing at full sending rate would now be unwise, as this might only increase the congestion. Although it is not guaranteed that all packets of the TCP connection take the same way through the network, this assumption holds for most of the packets. To mitigate congestion, TCP slows down the transmission rate dramatically. All other TCP connections experiencing the same congestion do exactly the same so the congestion is soon resolved. This cooperation of TCP connections in the internet is one of the main reasons for its survival as it is today.

Slow start

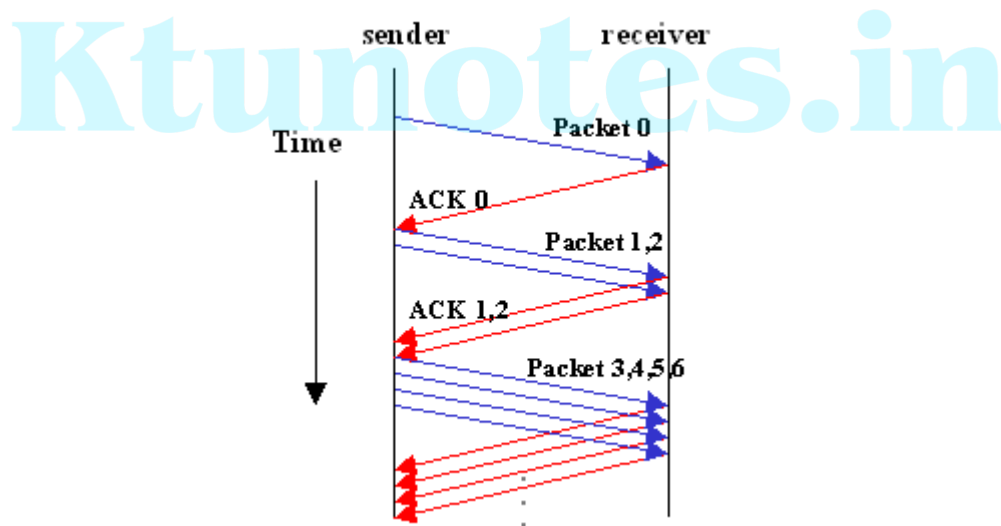


Fig.4.6 Slow start

TCP's reaction to a missing acknowledgement is quite drastic, but it is necessary to get rid of congestion quickly. The sender always calculates a **congestion window** for a receiver. The start size of the congestion window is one segment (TCP packet). The sender sends one packet and waits for acknowledgement. If this acknowledgement arrives, the sender increases the congestion window by one, now sending two packets (congestion window = 2).

After arrival of the two corresponding acknowledgements, the sender again adds 2 to the congestion window, one for each of the acknowledgements. Now the congestion window equals 4. This scheme doubles the congestion window every time the acknowledgements come back, which takes one round trip time (RTT). This is called the exponential growth of the congestion window in the slow start mechanism.

It is too dangerous to double the congestion window each time because the steps might become too large. The exponential growth stops at the **congestion threshold**. As soon as the congestion window reaches the congestion threshold, further increase of the transmission rate is only linear by adding 1 to the congestion window each time the acknowledgements come back.

Linear increase continues until a time-out at the sender occurs due to a missing acknowledgement, or until the sender detects a gap in transmitted data because of continuous acknowledgements for the same packet. In either case the sender sets the congestion threshold to half of the current congestion window. The congestion window itself is set to one segment and the sender starts sending a single segment. The exponential growth starts once more up to the new congestion threshold, then the window grows in linear fashion.

Fast retransmit/fast recovery

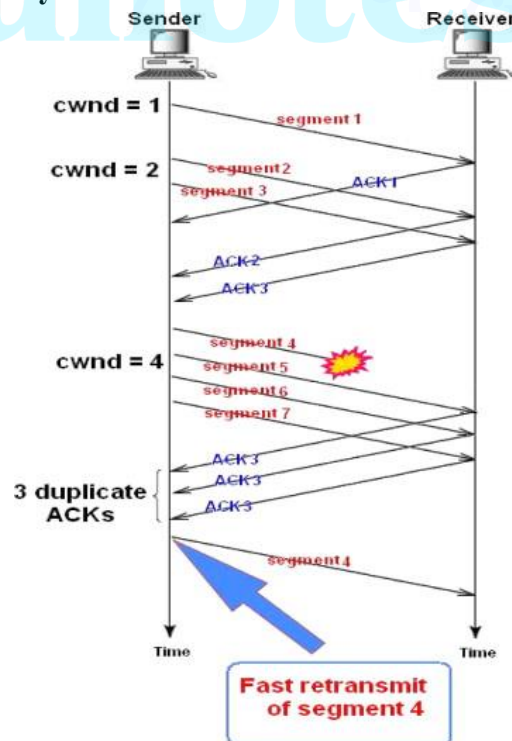


Fig.4.7 Fast retransmit/fast recovery

Two things lead to a reduction of the congestion threshold. One is a sender receiving continuous acknowledgements for the same packet. This informs the sender of two things. One is that the receiver got all packets up to the acknowledged packet in sequence. In TCP, a receiver sends acknowledgements only if it receives any packets from the sender. Receiving acknowledgements from a receiver also shows that the receiver continuously receives something from the sender. The gap in the packet stream is not due to severe congestion, but a simple packet loss due to a transmission error. The sender can now retransmit the missing packet(s) before the timer expires. This behavior is called **fast retransmit**

The receipt of acknowledgements shows that there is no congestion to justify a slow start. The sender can continue with the current congestion window. The sender performs a **fast recovery** from the packet loss. This mechanism can improve the efficiency of TCP dramatically.

The other reason for activating slow start is a time-out due to a missing acknowledgement. TCP using fast retransmit/fast recovery interprets this congestion in the network and activates the slow start mechanism.

Implications on mobility

Mobility itself can cause packet loss. There are many situations where a soft handover from one access point to another is not possible for a mobile end-system. For example, when using mobile IP, there could still be some packets in transit to the old foreign agent while the mobile node moves to the new foreign agent. The old foreign agent may not be able to forward those packets to the new foreign agent or even buffer the packets if disconnection of the mobile node takes too long. This packet loss has nothing to do with wireless access but is caused by the problems of rerouting traffic.

The TCP mechanism detecting missing acknowledgements via time-outs and concluding packet loss due to congestion cannot distinguish between the different causes. However, one cannot change TCP completely just to support mobile users or wireless links. The same arguments that were given to keep IP unchanged also apply to TCP. The installed base of computers using TCP is too large to be changed and, more important, mechanisms such as slow start keep the internet operable. Every enhancement to TCP, therefore, has to remain compatible with the standard TCP and must not jeopardize the cautious behavior of TCP in case of congestion. The following sections present some classical solutions before discussing current TCP tuning recommendations.

Classical TCP improvements

Indirect TCP

Two competing insights led to the development of indirect TCP (I-TCP). One is that TCP performs poorly together with wireless links; the other is that TCP within the fixed network cannot be changed. I-TCP segments a TCP connection into a fixed part and a wireless part. Figure 4.8 shows an example with a mobile host connected via a wireless link and an access point to the 'wired' internet where the correspondent host resides. The correspondent node could also use wireless access. The following would then also be applied to the access link of the correspondent host. Standard TCP is used between the fixed computer and the access point. No computer in the internet recognizes any changes to TCP. Instead of the mobile host, the access point now terminates the standard TCP connection, acting as a proxy. This means that the access point is now seen as the mobile host for the fixed host and as the fixed host for the mobile host. Between the access point and the mobile host, a special TCP, adapted to wireless links, is used. However, changing TCP for the wireless link is not a requirement. Even an unchanged TCP can benefit from the much shorter round trip time, starting retransmission much faster. A good place for segmenting the connection between mobile host and correspondent host is at the foreign agent of mobile IP (see chapter 8). The foreign agent controls the mobility of the mobile host anyway and can also hand over the connection to the next foreign agent when the mobile host moves on. However, one can also imagine separating the TCP connections at a special server.

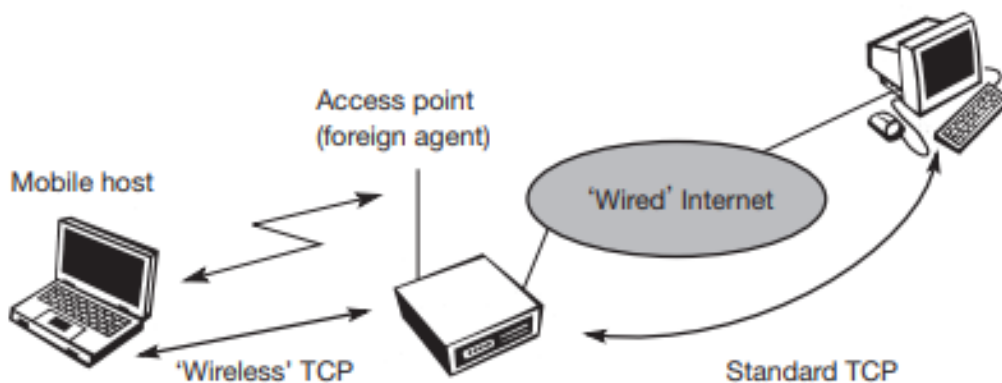


Fig.4.8 Indirect TCP

Advantages with I-TCP

1. I-TCP does not require any changes in the TCP protocol as used by the hosts in the fixed network or other hosts in a wireless network

2. Due to the strict partitioning into two connections, transmission errors on the wireless link, i.e., lost packets, cannot propagate into the fixed network
3. Optimizing of these new mechanisms is quite simple because they only cover one single hop
4. Only a Small delay between the mobile host and foreign agent

Disadvantages of I-TCP

1. The loss of the end-to-end semantics of TCP might cause problems if the foreign agent partitioning the TCP connection crashes
2. The handover latency may be much more problematic
3. The foreign agent must be a trusted entity

Snooping TCP

One of the drawbacks of I-TCP is the segmentation of the single TCP connection into two TCP connections. This loses the original end-to-end TCP semantic. The following TCP enhancement works completely transparently and leaves the TCP end-to-end connection intact. The main function of the enhancement is to buffer data close to the mobile host to perform fast local retransmission in case of packet loss. A good place for the enhancement of TCP could be the foreign agent in the Mobile IP context (see Figure 4.9).

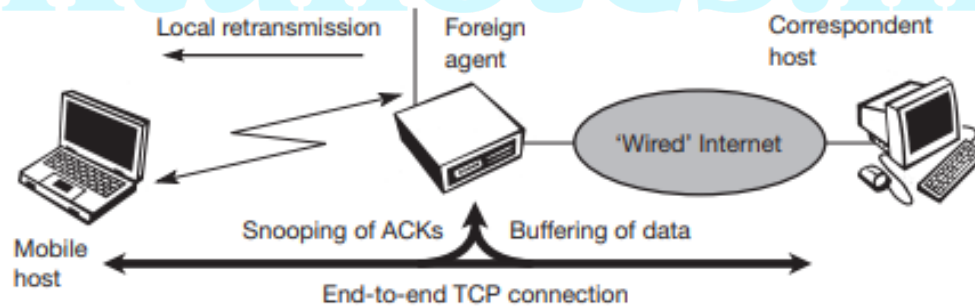


Fig.4.9 Snooping TCP

In this approach, the foreign agent buffers all packets with **destination mobile host** and additionally ‘snoops’ the packet flow in both directions to recognize acknowledgements. The reason for buffering packets toward the mobile node is to enable the foreign agent to perform a local retransmission in case of packet loss on the wireless link. The foreign agent buffers every packet until it receives an acknowledgement from the mobile host. If the foreign agent does not receive an acknowledgement from the mobile host within a certain amount of time,

either the packet or the acknowledgement has been lost. Alternatively, the foreign agent could receive a duplicate ACK which also shows the loss of a packet. Now the foreign agent retransmits the packet directly from the buffer, performing a much faster retransmission compared to the correspondent host. The time out for acknowledgements can be much shorter, because it reflects only the delay of one hop plus processing time.

To remain transparent, the foreign agent must not acknowledge data to the correspondent host. This would make the correspondent host believe that the mobile host had received the data and would violate the end-to-end semantic in case of a foreign agent failure. However, the foreign agent can filter the duplicate acknowledgements to avoid unnecessary retransmissions of data from the correspondent host. If the foreign agent now crashes, the time-out of the correspondent host still works and triggers a retransmission. The foreign agent may discard duplicates of packets already retransmitted locally and acknowledged by the mobile host. This avoids unnecessary traffic on the wireless link.

Data transfer from the mobile host with **destination correspondent host** works as follows. The foreign agent snoops into the packet stream to detect gaps in the sequence numbers of TCP. As soon as the foreign agent detects a missing packet, it returns a negative acknowledgement (NACK) to the mobile host. The mobile host can now retransmit the missing packet immediately. Reordering of packets is done automatically at the correspondent host by TCP.

Advantages of snooping TCP

1. The end-to-end TCP semantic is preserved
2. The correspondent host does not need to be changed; most of the enhancements are in the foreign agent
3. It does not need a handover of state as soon as the mobile host moves to another foreign agent

Disadvantages of snooping TCP

1. Snooping TCP does not isolate the behavior of the wireless link as well as I-TCP
2. Using negative acknowledgements between the foreign agent and the mobile host assumes additional mechanisms on the mobile host.
3. All efforts for snooping and buffering data may be useless if certain encryption schemes are applied end-to-end between the correspondent host and mobile host.

Mobile TCP

The **M-TCP (mobile TCP)** approach has the same goals as I-TCP and snooping TCP: to prevent the sender window from shrinking if bit errors or disconnection but not congestion cause current problems. M-TCP wants to improve overall throughput, to lower the delay, to maintain end-to-end semantics of TCP, and to provide a more efficient handover. Additionally, M-TCP is especially adapted to the problems arising from lengthy or frequent disconnections

M-TCP splits the TCP connection into two parts as I-TCP does. An unmodified TCP is used on the standard host-**supervisory host (SH)** connection, while an optimized TCP is used on the SH-MH connection. The supervisory host is responsible for exchanging data between both parts similar to the proxy in I-TCP (see Figure 4.8). The M-TCP approach assumes a relatively low bit error rate on the wireless link. Therefore, it does not perform caching/retransmission of data via the SH. If a packet is lost on the wireless link, it has to be retransmitted by the original sender. This maintains the TCP end-to-end semantics.

The SH monitors all packets sent to the MH and ACKs returned from the MH. If the SH does not receive an ACK for some time, it assumes that the MH is disconnected. It then chokes the sender by setting the sender's window size to 0. Setting the window size to 0 forces the sender to go into **persistent mode**, i.e., the state of the sender will not change no matter how long the receiver is disconnected. This means that the sender will not try to retransmit data. As soon as the SH (either the old SH or a new SH) detects connectivity again, it reopens the window of the sender to the old value. The sender can continue sending at full speed. This mechanism does not require changes to the sender's TCP.

The wireless side uses an adapted TCP that can recover from packet loss much faster. This modified TCP does not use slow start, thus, M-TCP needs a **bandwidth manager** to implement fair sharing over the wireless link.

Advantages of M-TCP

- It maintains the TCP end-to-end semantics. The SH does not send any ACK itself but forwards the ACKs from the MH.
- If the MH is disconnected, it avoids useless retransmissions, slow starts or breaking connections by simply shrinking the sender's window to 0.
- Since it does not buffer data in the SH as I-TCP does, it is not necessary to forward buffers to a new SH.

Disadvantages of M-TCP:

- As the SH does not act as proxy as in I-TCP, packet loss on the wireless link due to bit errors is propagated to the sender. M-TCP assumes low bit error rates, which is not always a valid assumption.
- A modified TCP on the wireless link not only requires modifications to the MH protocol software but also new network elements like the bandwidth manager.

Fast retransmit/fast recovery

As soon as the mobile host registers at a new foreign agent using mobile IP, it starts sending duplicated acknowledgements to correspondent hosts. The proposal is to send three duplicates. This forces the corresponding host to go into fast retransmit mode and not to start slow start, i.e., the correspondent host continues to send with the same rate it did before the mobile host moved to another foreign agent. As the mobile host may also go into slow start after moving to a new foreign agent, this approach additionally puts the mobile host into fast retransmit. The mobile host retransmits all unacknowledged packets using the current congestion window size without going into slow start.

The **advantage** of this approach is its simplicity. Only minor changes in the mobile host's software already result in a performance increase. No foreign agent or correspondent host has to be changed.

The main **disadvantage** of this scheme is the insufficient isolation of packet losses. Forcing fast retransmission increases the efficiency, but retransmitted packets still have to cross the whole network between correspondent host and mobile host. The approach focuses on loss due to handover: packet loss due to problems on the wireless link is not considered. This approach requires more cooperation between the mobile IP and TCP layer making it harder to change one without influencing the other.

Transmission/time-out freezing

The use of mobile hosts in a car driving into a tunnel, which loses its connection to, e.g., a satellite (however, many tunnels and subways provide connectivity via a mobilephone), or a user moving into a cell with no capacity left over. In this case, the mobile phone system will interrupt the connection. The reaction of TCP, even with the enhancements of above, would be a disconnection after a time out. Quite often, the MAC layer has already noticed connection problems, before the connection is actually interrupted from a TCP point of view. Additionally, the MAC layer knows the real reason for the interruption and does

not assume congestion, as TCP would. The MAC layer can inform the TCP layer of an upcoming loss of connection or that the current interruption is not caused by congestion. TCP can now stop sending and ‘freezes’ the current state of its congestion window and further timers. If the MAC layer notices the upcoming interruption early enough, both the mobile and correspondent host can be informed. With a fast interruption of the wireless link, additional mechanisms in the access point are needed to inform the correspondent host of the reason for interruption. Otherwise, the correspondent host goes into slow start assuming congestion and finally breaks the connection. As soon as the MAC layer detects connectivity again, it signals TCP that it can resume operation at exactly the same point where it had been forced to stop.

The **advantage** of this approach is that it offers a way to resume TCP connections even after longer interruptions of the connection. It is independent of any other TCP mechanism, such as acknowledgements or sequence numbers, so it can be used together with encrypted data. However, this scheme has some severe **disadvantages**. Not only does the software on the mobile host have to be changed, to be more effective the correspondent host cannot remain unchanged. All mechanisms rely on the capability of the MAC layer to detect future interruptions. Freezing the state of TCP does not help in case of some encryption schemes that use time-dependent random numbers.

Selective retransmission

A very useful extension of TCP is the use of selective retransmission. TCP acknowledgements are cumulative, i.e., they acknowledge in-order receipt of packets up to a certain packet. If a single packet is lost, the sender has to retransmit everything starting from the lost packet (go-back-n retransmission). This obviously wastes bandwidth, not just in the case of a mobile network, but for any network. TCP can indirectly request a selective retransmission of packets. The receiver can acknowledge single packets, not only trains of in-sequence packets. The sender can now determine precisely which packet is needed and can retransmit it.

The **advantage** of this approach is obvious: a sender retransmits only the lost packets. This lowers bandwidth requirements and is extremely helpful in slow wireless links. However, there might be the minor **disadvantage** of more complex software on the receiver side, because now more buffer is necessary to resequence data and to wait for gaps to be filled.

Transaction-oriented TCP

Assume an application running on the mobile host that sends a short request to a server from time to time, which responds with a short message. If the application requires reliable transport of the packets, it may use TCP. Using TCP now requires several packets over the wireless link. First, TCP uses a three-way handshake to establish the connection. At least one additional packet is usually needed for transmission of the request, and requires three more packets to close the connection via a three-way handshake. Assuming connections with a lot of traffic or with a long duration, this overhead is minimal. But in an example of only one data packet, TCP may need seven packets altogether.

This led to the development of a transaction-oriented TCP T/TCP can combine packets for connection establishment and connection release with user data packets. This can reduce the number of packets down to two instead of seven. Similar considerations led to the development of a transaction service in WAP

The obvious **advantage** for certain applications is the reduction in the overhead which standard TCP has for connection setup and connection release. However, T/TCP is not the original TCP anymore, so it requires changes in the mobile host and all correspondent hosts, which is a major **disadvantage**.

Approach	Mechanism	Advantages	Disadvantages
Indirect TCP	Splits TCP connection into two connections	Isolation of wireless link, simple	Loss of TCP semantics, higher latency at handover, security problems
Snooping TCP	Snoops data and acknowledgements, local retransmission	Transparent for end-to-end connection, MAC integration possible	Insufficient isolation of wireless link, security problems
M-TCP	Splits TCP connection, chokes sender via window size	Maintains end-to-end semantics, handles long term and frequent disconnections	Bad isolation of wireless link, processing overhead due to bandwidth management, security problems
Fast retransmit/ fast recovery	Avoids slow-start after roaming	Simple and efficient	Mixed layers, not transparent
Transmission/ time-out freezing	Freezes TCP state at disconnection, resumes after reconnection	Independent of content, works for longer interruptions	Changes in TCP required, MAC dependent
Selective retransmission	Retransmits only lost data	Very efficient	Slightly more complex receiver software, more buffer space needed
Transaction-oriented TCP	Combines connection setup/release and data transmission	Efficient for certain applications	Changes in TCP required, not transparent, security problems

Wireless application protocol (version 1.x)

The basic objectives of the WAP Forum and now of the OMA are to bring diverse internet content (e.g., web pages, push services) and other data services (e.g., stock quotes) to digital cellular phones and other wireless, mobile terminals (e.g., PDAs, laptops). Moreover, a protocol suite should enable global wireless communication across different wireless network technologies, e.g., GSM, CDPD, UMTS etc. The forum is embracing and extending existing standards and technologies of the internet wherever possible and is creating a framework for the development of contents and applications that scale across a very wide range of wireless bearer networks and wireless device types.

All solutions must be:

- **Interoperable**, i.e., allowing terminals and software from different vendors to communicate with networks from different providers;
- **Scalable**, i.e., protocols and services should scale with customer needs and number of customers;
- **Efficient**, i.e., provision of QoS suited to the characteristics of the wireless and mobile networks;
- **Reliable**, i.e., provision of a consistent and predictable platform for deploying services;
- **Secure**, i.e., preservation of the integrity of user data, protection of devices and services from security problems.

Architecture

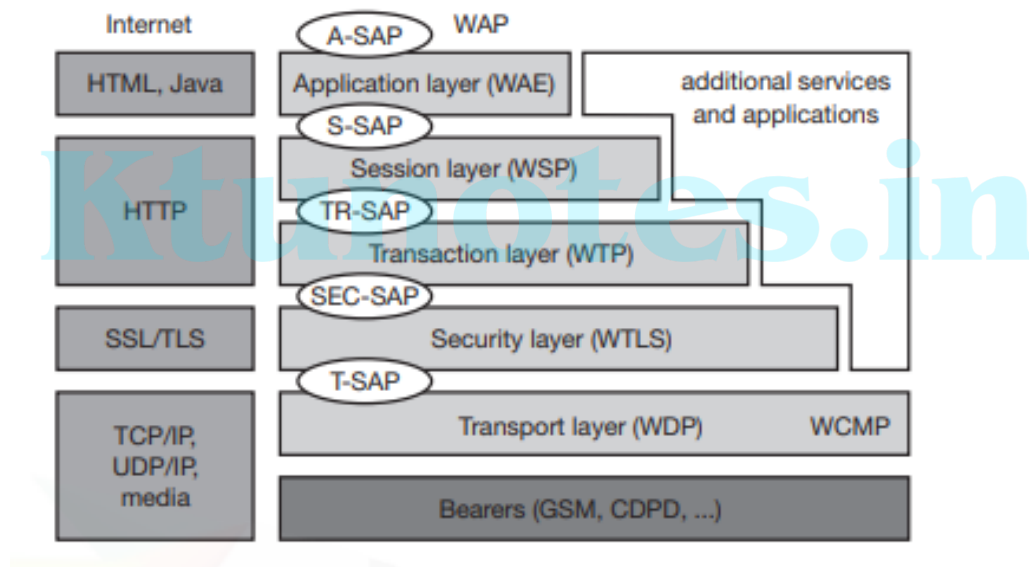


Fig.4.10 WAP architecture

Figure 4.10 gives an overview of the WAP architecture, its protocols and components, and compares this architecture with the typical internet architecture when using the world wide web. This comparison is often cited by the WAP Forum and it helps to understand the architecture. This comparison can be misleading as not all components and protocols shown at the same layer are comparable. For consistency reasons with the existing specification, the following stays with the model as shown in Figure 4.10.

The basis for transmission of data is formed by different **bearer services**. WAP

does not specify bearer services, but uses existing data services and will integrate further services. Examples are message services, such as short message service (SMS) of GSM, circuit-switched data, such as high-speed circuit switched data (HSCSD) in GSM, or packet switched data, such as general packet radio service (GPRS) in GSM. Many other bearers are supported, such as CDPD, IS-136, PHS. No special interface has been specified between the bearer service and the next higher layer, the **transport layer** with its **wireless datagram protocol (WDP)** and the additional **wireless control message protocol (WCMP)**, because the adaptation of these protocols are bearer-specific. The transport layer offers a bearer independent, consistent datagram-oriented service to the higher layers of the WAP architecture. Communication is done transparently over one of the available bearer services. The **transport layer service access point (T-SAP)** is the common interface to be used by higher layers independent of the underlying network.

The next higher layer, the **security layer** with its **wireless transport layer security protocol WTLS** offers its service at the **security SAP (SEC-SAP)**. WTLS is based on the transport layer security (TLS, formerly SSL, secure sockets layer) already known from the www. WTLS has been optimized for use in wireless networks with narrow-band channels. It can offer data integrity, privacy, authentication, and (some) denial-of-service protection.

The WAP **transaction layer** with its **wireless transaction protocol (WTP)** offers a lightweight transaction service at the **transaction SAP (TR-SAP)**. This service efficiently provides reliable or unreliable requests and asynchronous transactions. Tightly coupled to this layer is the next higher layer, if used for connection-oriented service. The **session layer** with the **wireless session protocol (WSP)** currently offers two services at the **session-SAP (S-SAP)**, one connection-oriented and one connectionless if used directly on top of WDP. A special service for browsing the web (WSP/B) has been defined that offers HTTP/1.1 functionality, long-lived session state, session suspend and resume, session migration and other features needed for wireless mobile access to the web.

Finally the **application layer** with the **wireless application environment (WAE)** offers a framework for the integration of different www and mobile telephony applications. It offers many protocols and services with special service access points. The main issues here are scripting languages, special markup languages, interfaces to telephony applications, and many content formats adapted to the special requirements of small, handheld, wireless devices.

Figure 4.10 not only shows the overall WAP architecture, but also its relation to the traditional internet architecture for www applications. The WAP transport layer

together with the bearers can be (roughly) compared to the services offered by TCP or UDP over IP and different media in the internet. If a bearer in the WAP architecture already offers IP services then UDP is used as WDP. The TLS/SSL layer of the internet has also been adopted for the WAP architecture with some changes required for optimization. The functionality of the session and transaction layer can roughly be compared with the role of HTTP in the web architecture. However, HTTP does not offer all the additional mechanisms needed for efficient wireless, mobile access (e.g., session migration, suspend/resume). Finally, the application layer offers similar features as HTML and Java. Again, special formats and features optimized for the wireless scenario have been defined and telephony access has been added.

Different scenarios are possible for the integration of WAP components into existing wireless and fixed networks. On the left side, different fixed networks, such as the traditional internet and the public switched tele- phone network (PSTN), are shown. One cannot change protocols and services of these existing networks so several new elements will be implemented between these networks and the WAP-enabled wireless, mobile devices in a wireless network on the right-hand side.

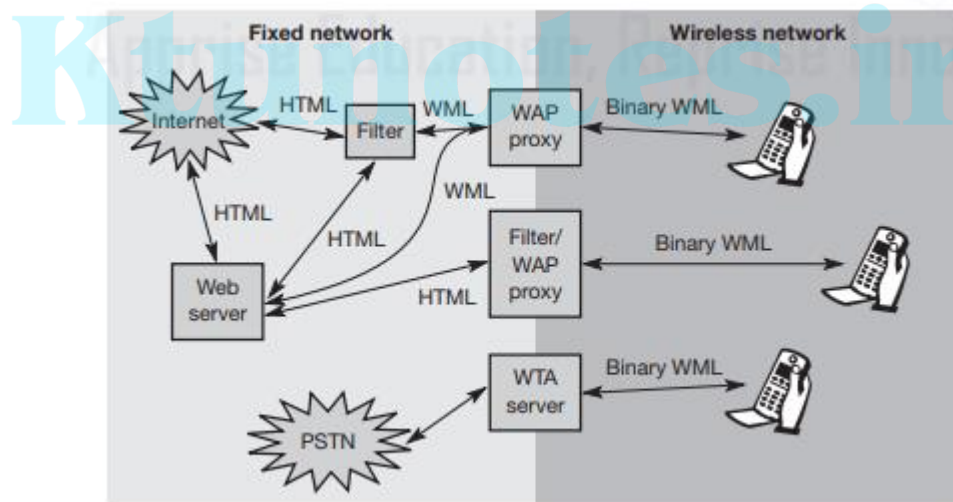


Fig.4.11 Examples for the integration of WAP components

The current www in the internet offers web pages with the help of HTML and web servers. To be able to browse these pages or additional pages with hand-held devices, a wireless markup language (WML) has been defined in WAP. Special filters within the fixed network can now translate HTML into WML, web servers can already provide pages in WML, or the gateways between the fixed and wireless network can translate HTML into WML. These gateways not only filter pages but

also act as proxies for web access, as explained in the following sections. WML is additionally converted into binary WML for more efficient transmission.

In a similar way, a special gateway can be implemented to access traditional telephony services via binary WML. This wireless telephony application (WTA) server translates, e.g., signaling of the telephone network (incoming call etc.) into WML events displayed at the handheld device. It is important to notice the integrated view for the wireless client of all different services, telephony and web, via the WAE.

Wireless datagram protocol

The **wireless datagram protocol (WDP)** operates on top of many different bearer services capable of carrying data. At the T-SAP WDP offers a consistent datagram transport service independent of the underlying bearer. To offer this consistent service, the adaptation needed in the transport layer can differ depending on the services of the bearer. The closer the bearer service is to IP, the smaller the adaptation can be. If the bearer already offers IP services, UDP is used as WDP. WDP offers more or less the same services as UDP.

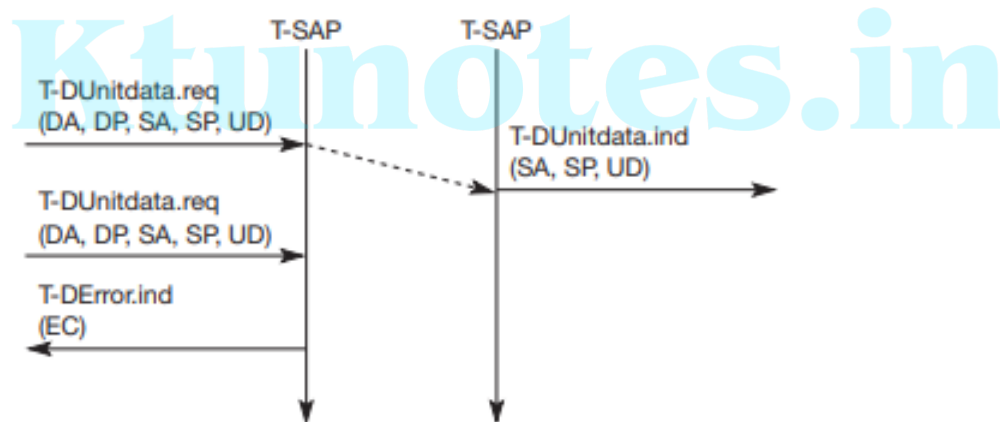


Fig.4.12 WDP service primitives

WDP offers **source and destination port numbers** used for multiplexing and demultiplexing of data respectively. The service primitive to send a datagram is **T-DUnitdata.req** with the **destination address (DA)**, **destination port (DP)**, **Source address (SA)**, **source port (SP)**, and **user data (UD)** as mandatory parameters (see Figure 4.12). Destination and source address are unique addresses for the receiver and sender of the user data. These could be MSISDNs (i.e., a telephone number), IP addresses, or any other unique identifiers. The **T-DUnitdata.ind** service

primitive indicates the reception of data. Here destination address and port are only optional parameters.

If a higher layer requests a service the WDP cannot fulfill, this error is indicated with the **T-DError.ind** service primitive as shown in Figure 4.12. An **error code (EC)** is returned indicating the reason for the error to the higher layer. It is only allowed to use the primitive to indicate local problems, such as a user data size that is too large.

If any errors happen when WDP datagrams are sent from one WDP entity to another (e.g. the destination is unreachable, no application is listening to the specified destination port etc.), the **wireless control message protocol (WCMP)** provides error handling mechanisms for WDP and should therefore be implemented. WCMP contains control messages that resemble the internet control message protocol (ICMP) for IPv4, (Conta, 1998) for IPv6) messages and can also be used for diagnostic and informational purposes. WCMP can be used by WDP nodes and gateways to report errors. However, WCMP error messages must not be sent as response to other WCMP error messages. In IP-based networks, ICMP will be used as WCMP (e.g., CDPD, GPRS). Typical WCMP messages are **destination unreachable** (route, port, address unreachable), **parameter problem** (errors in the packet header), **message too big**, **reassembly failure**, or **echo request/reply**.

An additional **WDP management entity** supports WDP and provides information about changes in the environment, which may influence the correct operation of WDP. Important information is the current configuration of the device, currently available bearer services, processing and memory resources etc. Design and implementation of this management component is considered vendor-specific and is outside the scope of WAP.

If the bearer already offers IP transmission, WDP relies on the segmentation (called fragmentation in the IP context) and reassembly capabilities of the IP layer as specified in. Otherwise, WDP has to include these capabilities, which is, e.g., necessary for the GSM SMS. The WAP specification provides many more adaptations to almost all bearer services currently available or planned for the future.

Wireless transport layer security

The **wireless transport layer security (WTLS)**, can be integrated into the WAP architecture on top of WDP as specified in WTLS can provide different levels of security (for privacy, data integrity, and authentication) and has been optimized for low bandwidth, high-delay bearer networks. WTLS takes into account the low processing power and very limited memory capacity of the mobile devices for cryptographic algorithms. WTLS supports datagram and connection-

oriented transport layer protocols. WTLS took over many features and mechanisms from TLS (formerly SSL, secure sockets layer, but it has an optimized hand- shaking between the peers.

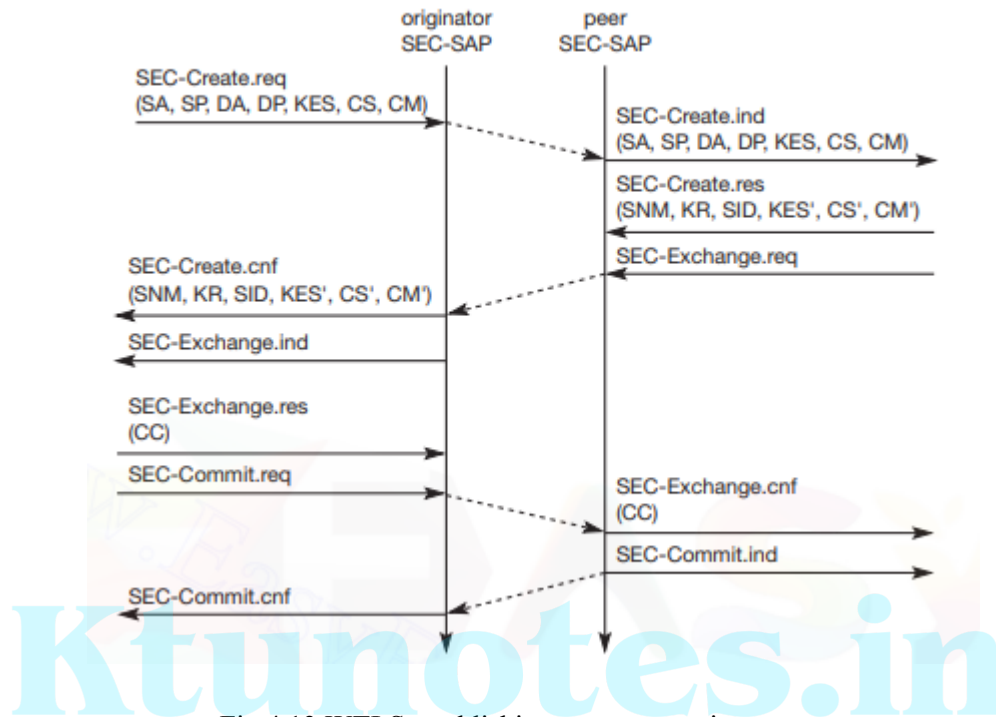


Fig.4.13 WTLS establishing a secure session

Before data can be exchanged via WTLS, a secure session has to be established. This session establishment consists of several steps: Figure 4.13 illustrates the sequence of service primitives needed for a so-called ‘full handshake’ (several optimizations are possible). The originator and the peer of the secure session can both interrupt session establishment any time, e.g., if the parameters proposed are not acceptable.

The first step is to initiate the session with the **SEC-Create** primitive. Parameters are **source address (SA)**, **source port (SP)** of the originator, **destination address (DA)**, **destination port (DP)** of the peer. The originator proposes a **key exchange suite (KES)**, a **cipher suite (CS)**, and a **compression method (CM)**. The peer answers with parameters for the **sequence number mode (SNM)**, the **key refresh cycle (KR)** (i.e., how often keys are refreshed within this secure session), the **session identifier (SID)** (which is unique with each peer), and the selected **key exchange suite (KES')**, **cipher suite (CS')**, **compression method (CM')**. The peer also issues a **SEC-Exchange** primitive. This indicates that the

peer wishes to perform public-key authentication with the client, i.e., the peer requests a **client certificate (CC)** from the originator.

The first step of the secure session creation, the negotiation of the security parameters and suites, is indicated on the originator's side, followed by the request for a certificate. The originator answers with its certificate and issues a **SEC-Commit.req** primitive. This primitive indicates that the handshake is completed for the originator's side and that the originator now wants to switch into the newly negotiated connection state. The certificate is delivered to the peer side and the SEC-Commit is indicated. The WTLS layer of the peer sends back a confirmation to the originator. This concludes the full handshake for secure session setup.

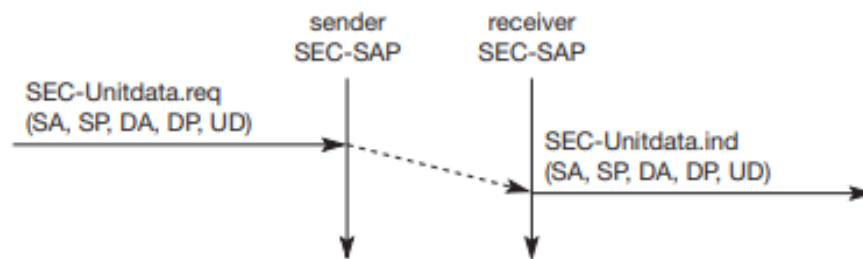


Fig.4.13 WTLS datagram transfer

After setting up a secure connection between two peers, user data can be exchanged. This is done using the simple **SEC-Unitdata** primitive as shown in Figure 4.13. SEC-Unitdata has exactly the same function as T-DUnitdata on the WDP layer, namely it transfers a datagram between a sender and a receiver. This data transfer is still unreliable, but is now secure. This shows that WTLS can be easily plugged into the protocol stack on top of WDP. The higher layers simply use SEC-Unitdata instead of T-DUnitdata. The parameters are the same here: **source address (SA)**, **source port (SP)**, **destination address (DA)**, **destination port (DP)**, and **user data (UD)**.

Although WTLS allows for different encryption mechanisms with different key lengths, it is quite clear that due to computing power on the handheld devices the encryption provided cannot be very strong. If applications require stronger security, it is up to an application or a user to apply stronger encryption on top of the whole protocol stack and use WTLS as a basic security level only. Many programs are available for this purpose. It is important to note that the security association in WTLS exists between the mobile WAP-enabled device and a WAP server or WAP gateway only. If an application accesses another server via the gateway, additional mechanisms are needed for end-to-end security.

Wireless transaction protocol

The **wireless transaction protocol (WTP)** is on top of either WDP or, if security is required, WTLS. WTP has been designed to run on very thin clients, such as mobile phones. WTP offers several advantages to higher layers, including an improved reliability over datagram services, improved efficiency over connection-oriented services, and support for transaction-oriented services such as web browsing. In this context, a transaction is defined as a request with its response.

WTP offers many features to the higher layers. The basis is formed from three **classes of transaction service** as explained in the following paragraphs. Class 0 provides unreliable message transfer without any result message. Classes 1 and 2 provide reliable message transfer, class 1 without, class 2 with, exactly one reliable result message (the typical request/response case). WTP achieves reliability using **duplicate removal, retransmission, acknowledgements** and unique **transaction identifiers**. No WTP-class requires any connection set-up or tear-down phase. This avoids unnecessary overhead on the communication link. WTP allows for **asynchronous transactions, abort of transactions, concatenation of messages**, and can **report success or failure** of reliable messages.

To be consistent with the specification, in the following the term **initiator** is used for a WTP entity initiating a transaction (aka client), and the term **responder** for the WTP entity responding to a transaction (aka server). The three service primitives offered by WTP are **TR-Invoke** to initiate a new transaction, **TR-Result** to send back the result of a previously initiated transaction, and **TR-Abort** to abort an existing transaction. The PDUs exchanged between two WTP entities for normal transactions are the **invoke PDU, ack PDU, and result PDU**. The use of the service primitives, the PDUs, and the associated parameters with the classes of transaction service will be explained in the following sections.

A special feature of WTP is its ability to provide a **user acknowledgement** or, alternatively, an **automatic acknowledgement** by the WTP entity. If user acknowledgement is required, a WTP user has to confirm every message received by a WTP entity. A user acknowledgement provides a stronger version of a confirmed service because it guarantees that the response comes from the user of the WTP and not the WTP entity itself.

WTP class 0

Class 0 offers an unreliable transaction service without a result message. The transaction is stateless and cannot be aborted. The service is requested with the **TR-Invoke.req** primitive as shown in Figure 4.14. Parameters are the **source address**

(SA), source port (SP), destination address (DA), destination port (DP). Additionally, with the **A** flag the user of this service can determine, if the responder WTP entity should generate an **acknowledgement** or if a user acknowledgement should be used. The WTP layer will transmit the **user data (UD)** transparently to its destination. The class type **C** indicates here class 0. Finally, the transaction **handle H** provides a simple index to uniquely identify the transaction and is an alias for the tuple (SA, SP, DA, DP), i.e., a socket pair, with only local significance.

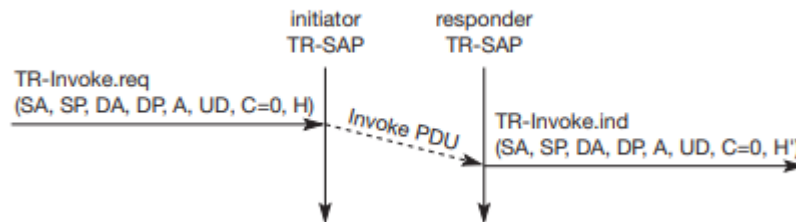


Fig.4.14 Basic transaction, WTP class 0

The WTP entity at the initiator sends an invoke PDU which the responder receives. The WTP entity at the responder then generates a **TR-Invoke.ind** primitive with the same parameters as on the initiator's side, except for H' which is now the local handle for the transaction on the responder's side.

In this class, the responder does not acknowledge the message and the initiator does not perform any retransmission. WTP class 0 augments the transaction service with a simple datagram like service for occasional use by higher layers.

WTP class 1

Class 1 offers a reliable transaction service but without a result message. Again, the initiator sends an invoke PDU after a **TR-Invoke.req** from a higher layer. This time, class equals '1', and no user acknowledgement has been selected as shown in Figure 4.15. The responder signals the incoming invoke PDU via the **TR-Invoke.ind** primitive to the higher layer and acknowledges automatically without user intervention. The specification also allows the user on the responder's side to acknowledge, but this acknowledgement is not required. For the initiator the transaction ends with the reception of the acknowledgement. The responder keeps the transaction state for some time to be able to retransmit the acknowledgement if it receives the same invoke PDU again indicating a loss of the acknowledgement. If a user of the WTP class 1 service on the initiator's side requests a user acknowledgement on the responder's side, the sequence diagram looks like Figure4.16. Now the WTP entity on the responder's side does not send

an acknowledgement automatically, but waits for the **TR-Invoke.res** service primitive from the user. This service primitive must have the appropriate local handle H' for identification of the right transaction. The WTP entity can now send the ack PDU. Typical uses for this transaction class are reliable push services.

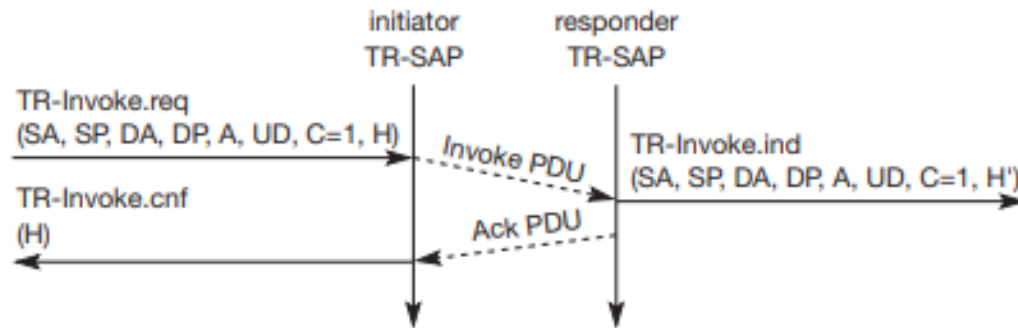


Fig.4.15 Basic transaction, WTP class 1, no user acknowledgement

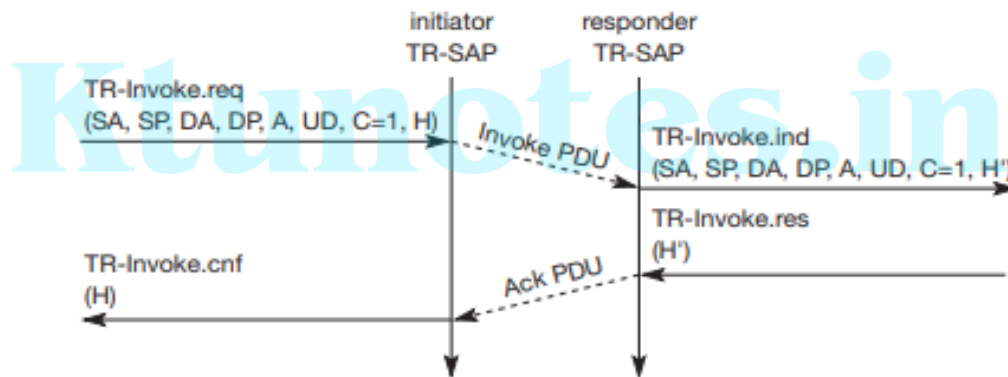


Fig.4.16 Basic transaction, WTP class 1, with user acknowledgement

WTP class 2

Class 2 transaction service provides the classic reliable request/response transaction known from many client/server scenarios. Depending on user requirements, many different scenarios are possible for initiator/responder interaction. If the calculation of the result takes some time, the responder can put the initiator on “hold on” to prevent a retransmission of the invoke PDU as the initiator might assume packet loss if no result is sent back within a certain timeframe. This

is shown in Figure 4.17. After a time-out, the responder automatically generates an acknowledgement for the Invoke PDU. This shows the initiator that the responder is still alive and currently busy processing the request. After more time, the result PDU can be sent to the initiator as already explained.

WTP provides many more features not explained here, such as concatenation and separation of messages, asynchronous transactions with up to 2^{15} transactions outstanding, i.e., requested but without result up to now, and segmentation/reassembly of messages.

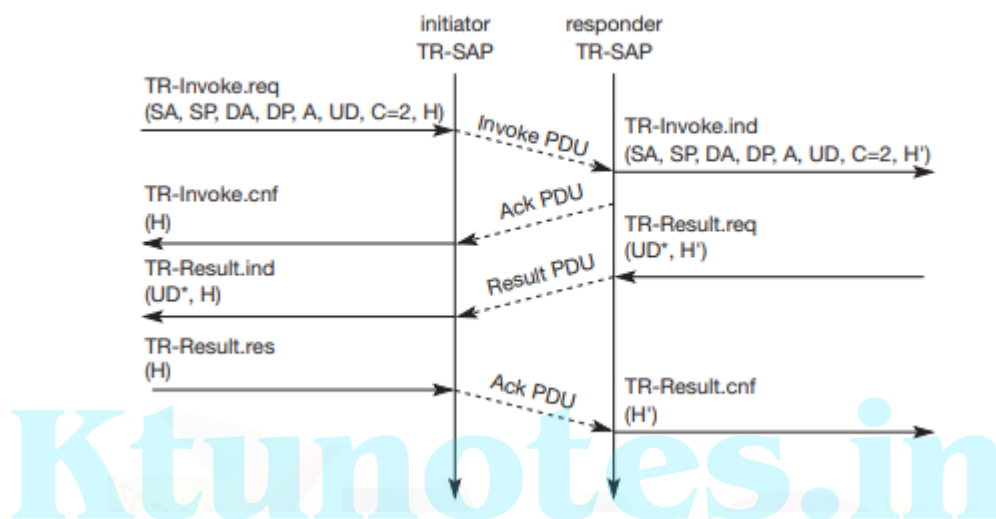


Fig.4.17 WTP class 2 transaction with “hold on”, no user acknowledgement

Wireless session protocol

The **wireless session protocol (WSP)** has been designed to operate on top of the datagram service WDP or the transaction service WTP. For both types, security can be inserted using the WTLS security layer if required. WSP provides a shared state between a client and a server to optimize content transfer. HTTP, a protocol WSP tries to replace within the wireless domain, is stateless, which already causes many problems in fixed networks. State is needed in web browsing, for example, to resume browsing in exactly the same context in which browsing has been suspended. This is an important feature for clients and servers. Client users can continue to work where they left the browser or when the network was interrupted, or users can get their customized environment every time they start the browser. Content providers can customize their pages to clients' needs and do not have to retransmit the same pages over and over again. WSP offers the following general features needed for content exchange between cooperating clients and servers:

- **Session management:** WSP introduces sessions that can be **established** from a client to a server and may be long lived. Sessions can also be **released** in an orderly manner. The capabilities of **suspending** and **resuming** a session are important to mobile applications. Assume a mobile device is being switched off – it would be useful for a user to be able to continue operation at exactly the point where the device was switched off. Session lifetime is independent of transport connection lifetime or continuous operation of a bearer network.

- **Capability negotiation:** Clients and servers can agree upon a common level of protocol functionality during session establishment. Example parameters to negotiate are maximum client SDU size, maximum outstanding requests, protocol options, and server SDU size.

- **Content encoding:** WSP also defines the efficient binary encoding for the content it transfers. WSP offers content typing and composite objects, as explained for web browsing. While WSP is a general-purpose session protocol, WAP has specified the **wireless session protocol/browsing (WSP/B)** which comprises protocols and services most suited for browsing-type applications. In addition to the general features of WSP, WSP/B offers the following features adapted to web browsing:

- **HTTP/1.1 functionality:** WSP/B supports the functions HTTP/1.1 offers, such as extensible request/reply methods, composite objects, and content type negotiation. WSP/B is a binary form of HTTP/1.1. HTTP/1.1 content headers are used to define content type, character set encoding, languages etc., but binary encodings are defined for well-known headers to reduce protocol overheads.

- **Exchange of session headers:** Client and server can exchange request/reply headers that remain constant over the lifetime of the session. These headers may include content types, character sets, languages, device capabilities, and other static parameters. WSP/B will not interpret header information but passes all headers directly to service users.

- **Push and pull data transfer:** Pulling data from a server is the traditional mechanism of the web. This is also supported by WSP/B using the request/response mechanism from HTTP/1.1. Additionally, WSP/B supports three push mechanisms for data transfer: a confirmed data push within an existing session context, a non-confirmed data push within an existing session context, and a non-confirmed data push without an existing session context.

- **Asynchronous requests:** Optionally, WSP/B supports a client that can send multiple requests to a server simultaneously. This improves efficiency for the requests and replies can now be coalesced into fewer messages. Latency is also improved, as each result can be sent to the client as soon as it is available.

As already mentioned, WSP/B can run over the transaction service WTP or the datagram service WDP. The following shows several protocol sequences typical for session management, method invocation, and push services.

WSP/B over WTP

WSP/B uses the three service classes of WTP. Class 0 is used for unconfirmed push, session resume, and session management. Confirmed push uses class 1, method invocation, session resume, and session management class 2. The following time sequence charts will give some examples.

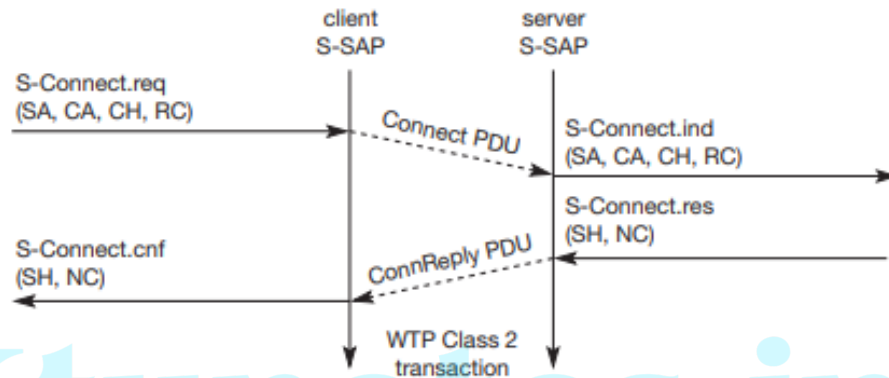


Fig.4.18 WSP/B session establishment

The above Figure 4.20 shows the session establishment of WSP/B using WTP class 2 transactions. With the **S-Connect.req** primitive, a client can request a new session. Parameters are the **server address (SA)**, the **client address (CA)**, and the optional **client header (CH)** and **requested capabilities (RC)**. The session layer directly uses the addressing scheme of the layer below. TR-SAP and S-SAP can be directly mapped. These headers can be used, e.g., for caching if they are constant throughout the session. Interpretation is up to the user of this service. The capabilities are needed for the capability negotiation between server and client as listed in the features of WSP above.

A very useful feature of WSP/B **session suspension** and **session resume** is shown in Figure 4.19. Session suspension will automatically abort all data transmission and freeze the current state of the session on the client and server side. A client suspends a session with **S-Suspend.req**, WTP transfers the **suspend PDU** to the server with a class 0 transaction, i.e., unconfirmed and unreliable. WSP/B will signal the suspension with **S-Suspend.ind** on the client and server side. The only parameter is the **reason R** for suspension. Reasons can be a user request or a suspension initiated by the service provider.

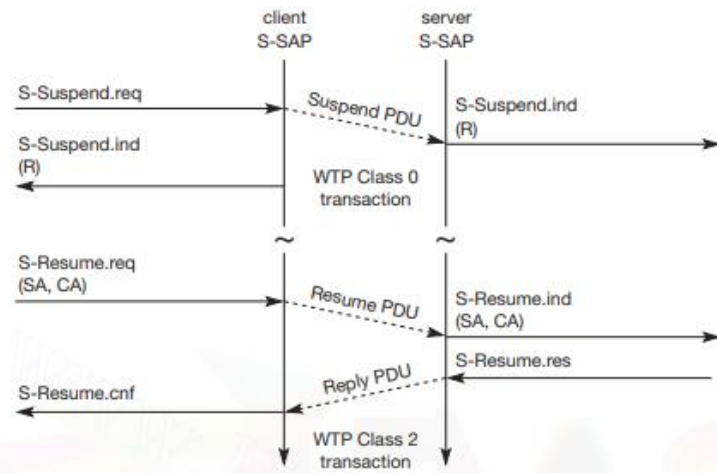


Fig.4.19 WSP/B session suspension and resume

As also shown in Figure 4.19, a client can later resume a suspended session with **S-Resume.req**. Parameters are **server address (SA)** and **client address (CA)**. If SA and CA are not the same as before suspending this session, it is the responsibility of the service user to map the addresses accordingly so that the same server instance will be contacted. Resuming a session is a confirmed operation. It is up to the server's operator how long this state is conserved.

Terminating a session is done by using the **S-Disconnect.req** service primitive (Figure 4.20). This primitive aborts all current method or push transactions used to transfer data. Disconnection is indicated on both sides using **S-Disconnect.ind**. The **reason R** for disconnection can be, e.g., network error, protocol error, peer request, congestion, and maximum SDU size exceeded. S-Disconnect.ind can also include parameters that redirect the session to another server where the session may continue.

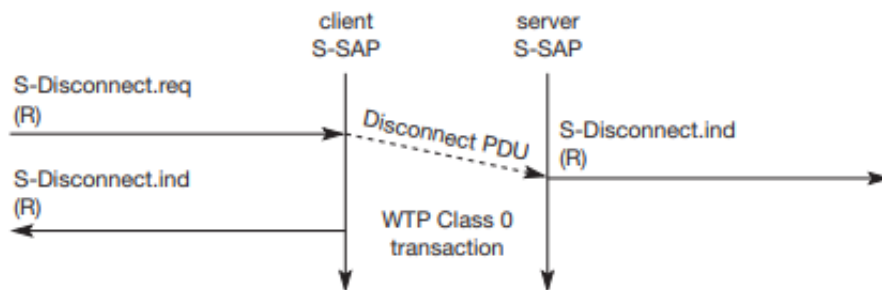


Fig.4.20 WSP/B session termination